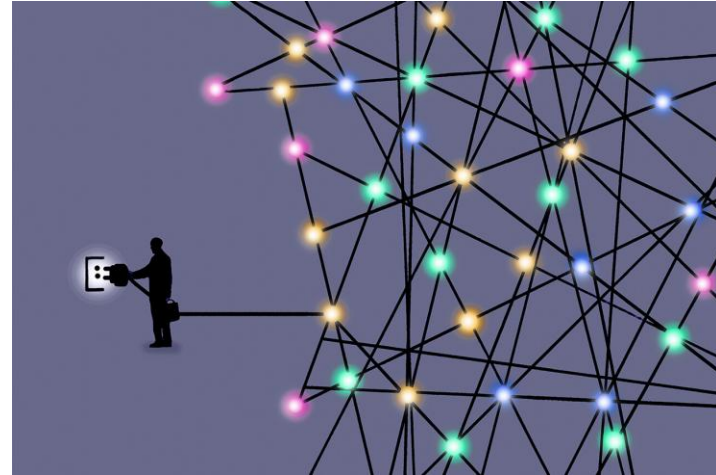
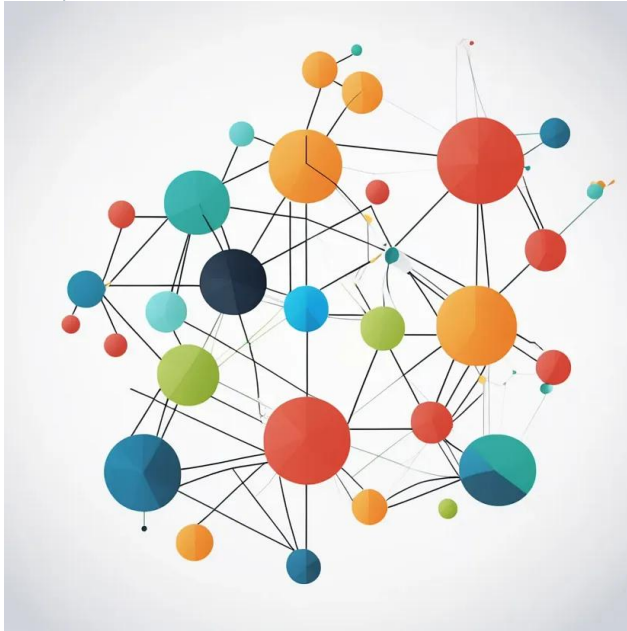


FIAP

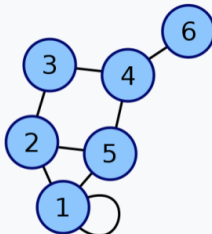
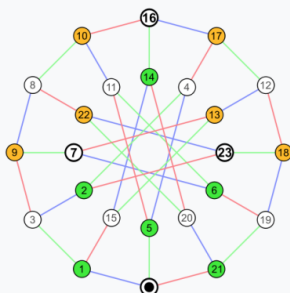
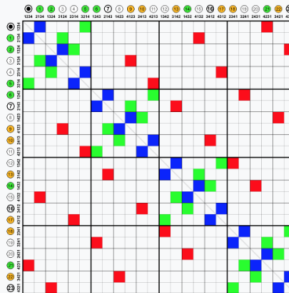
NABA



Grafos



Grafos

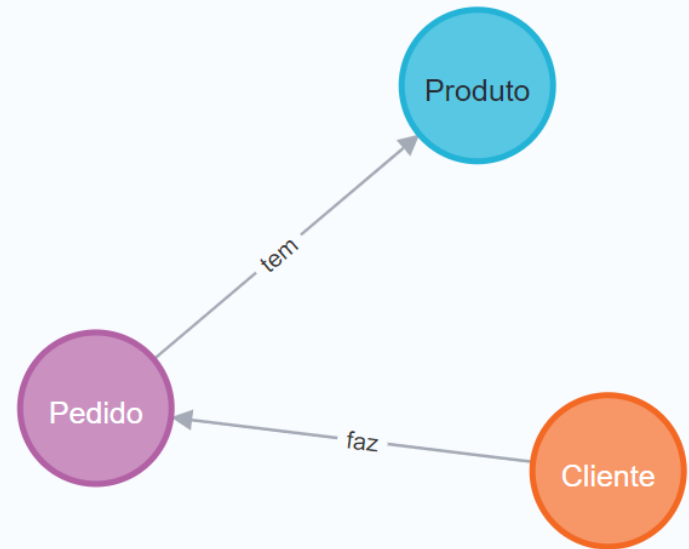
Labeled graph	Adjacency matrix
	$\begin{pmatrix} 2 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$ Coordinates are 1–6.
 Nauru graph	 Coordinates are 0–23. White fields are zeros, colored fields are ones.

Um **grafo** pode ser representado **como uma matriz** para diferentes finalidades, sendo armazenamento em **bancos de dados** ou visualização alternativa para análise, com o uso de bibliotecas complementares fornecendo **primitivas matemáticas** e algoritmos úteis.

Exercício Modelo Produtos

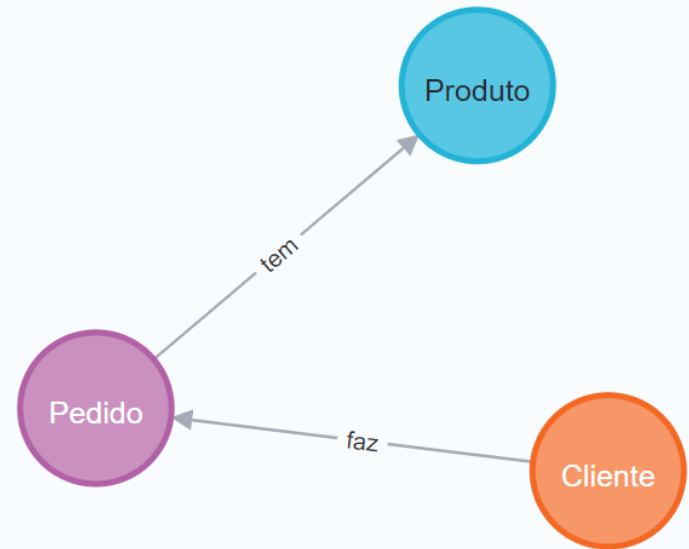
Modelo de grafos

nós, arestas e propriedades



Exercício Modelo Produtos

Vamos importar os dados a partir de arquivo .csv



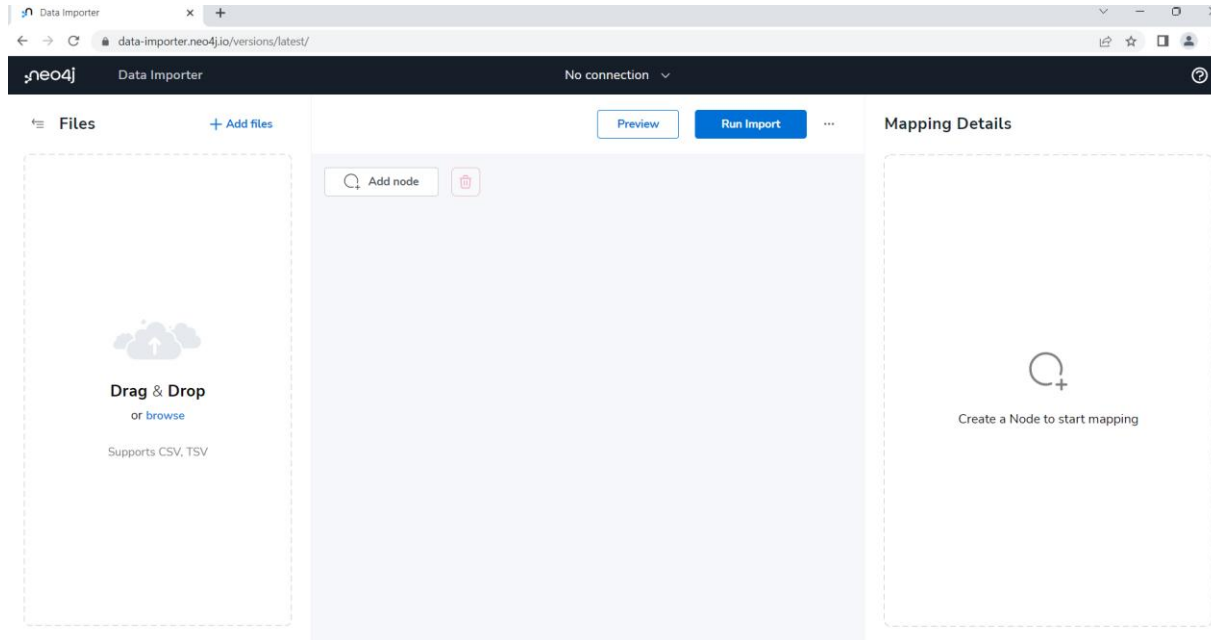
Exercício – Construindo grafos

- Os analista de dados receberam alguns arquivos csv
 - a) Verificar os dados no formato csv
 - b) Modelar os dados no formato grafo
 - c) Importar os dados no grafo
 - d) Explorar os dados – consultas para recomendação (Ex.: produto mais vendido por país, quem comprou também comprou, ...)

Utilizar o arquivo data.csv

Exercício – Construindo grafos

<https://data-importer.neo4j.io/>



Exercício – Construindo grafos

Modelo simplificado



Import results

Total time: 00:03:33

✓ **Run import completed**
Import completed successfully.

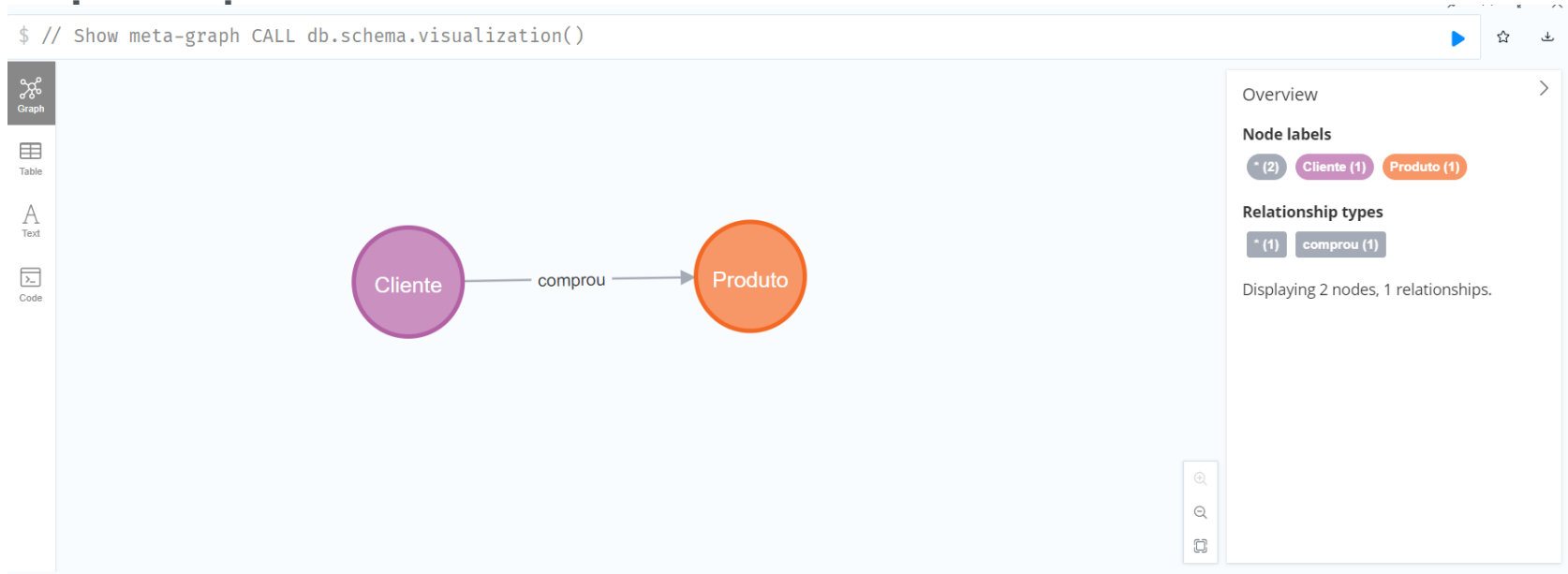
✓ **Produto** data.csv [Show Cypher](#)

Time taken	File size	File rows	Nodes created	Properties set	Labels added	Query count	Query time
00:01:16	43.5 MiB	541,909	4,070	100,922	4,070	46	00:01:13

✓ **Cliente** data.csv [Show Cypher](#)

Time taken	File size	File rows	Nodes created	Properties set	Labels added	Query count	Query time
00:00:20	43.5 MiB	541,909	4,372	4,372	4,372	46	00:00:19

Exercício – Construindo grafos



Exercício – Construindo grafos

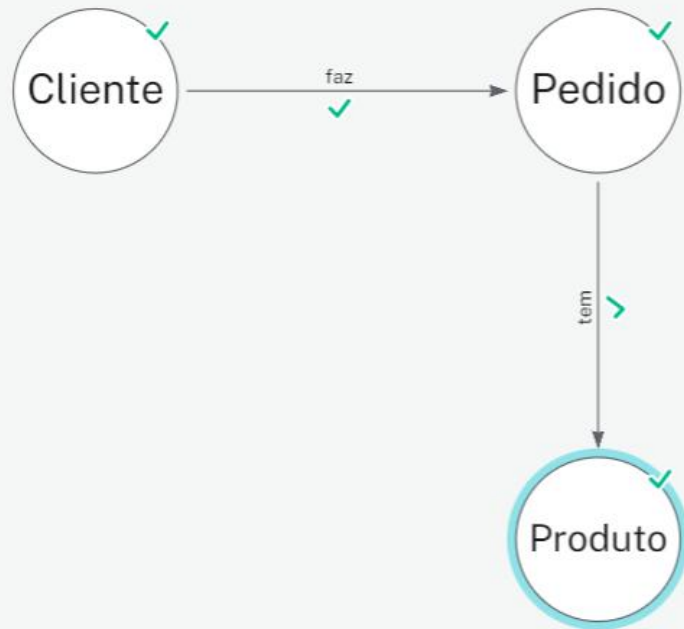
```

1 MATCH (p:Produto)←[r:comprou]-(c:Cliente)-[o:comprou]→(t:Produto)
2   where p.StockCode = "22262"
3   RETURN p.Description as produto, count(c) as qtdecli, t.Description as comproutb, count(t) as qtos, t.StockCode
4   order by qtos desc

```

	produto	qtdecli	comproutb	qtos	t.StockCode
1	"FELT EGG COSY CHICKEN"	39	"FELT EGG COSY WHITE RABBIT "	39	"22261"
2	"FELT EGG COSY CHICKEN"	26	"PACK OF 72 RETROSPOT CAKE CASES"	26	"21212"
3	"FELT EGG COSY CHICKEN"	23	"PAPER CHAIN KIT 50'S CHRISTMAS "	23	"22086"
4	"FELT EGG COSY CHICKEN"	22	"GINGERBREAD MAN COOKIE CUTTER"	22	"22966"
5	"FELT EGG COSY CHICKEN"	22	"NATURAL SLATE HEART CHALKBOARD "	22	"22457"
6	"FELT EGG COSY CHICKEN"	21	"PAPER CHAIN KIT EMPIRE"	21	"22084"

Exercício – Construindo grafos



Exercício – Construindo grafos

```
1 MATCH (c:Cliente)-[:faz]→(p:Pedido)-[t:tem]-(pro:Produto) WHERE c.CustomerID = 17850
2 RETURN p.InvoiceNo, pro.Description, t.Quantity order by p.InvoiceNo,pro.Description
```

	p.InvoiceNo	pro.Description	t.Quantity
10	536372	"HAND WARMER RED RETROSPOT"	6
11	536372	"HAND WARMER UNION JACK"	6
12	536373	"3 DRAWER ANTIQUE WHITE WOOD CABINET"	4
13	536373	"CREAM CUPID HEARTS COAT HANGER"	8

Exercício 1 – Construindo grafos

Construir algumas consultas **para recomendação de produtos** a partir:

- a) do produto mais vendido por categoria
- b) de um cliente qualquer
- c) de outro produto qualquer

***** Apresentar query e seus resultados *****

Graph Data Science

Algoritmos de ciência de dados para grafos calculam **distâncias** e caminhos, **similaridades** e **comunidades**; simulam os efeitos das mudanças no grafo; e permitem que sejam feitas previsões para **inferir novos nós** ou arestas ou **classificar** estruturas gráficas inteiras.

Graph Data Science




Pathfinding & Search

- Delta-Stepping Single-Source Shortest Path
- Dijkstra Source-Target Shortest Path
- Dijkstra Single-Source Shortest Path
- All Pairs Shortest Path
- A* Shortest Path
- Yen's Shortest Path
- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Random Walk
- Bellman-Ford Single-Source Shortest Path
- Minimum Weight Spanning Tree
- Minimum Directed Steiner Tree
- Prize-Collecting Steiner Tree
- Minimum Weight k-Spanning Tree
- Longest Path for DAG
- Collapse Path



Community Detection

- Triangle Count & Triangles Listing
- Local Clustering Coefficient
- Weakly Connected Components (WCC)
- Strongly Connected Components (SCC)
- Label Propagation
- Louvain
- K-1 Coloring
- Modularity Metric
- Modularity Optimization
- Speaker-Listener Label Propagation
- K-Means Clustering
- Leiden
- Approximate Maximum K-Cut
- Conductance Metric
- K-Core Decomposition
- HDBSCAN



Node Embeddings

- Fast Random Projection (FastRP)
- GraphSAGE
- Node2Vec
- Hash GNN - Knowledge Graph Embedding



Similarity

- Node Similarity
- Filtered Node Similarity
- K-Nearest Neighbors (KNN)
- Filtered KNN
- Similarity functions
 - Jaccard
 - Overlap
 - Cosine
 - Pearson
 - Euclidean Distance
 - Euclidean



Topological Link Prediction

- Adamic Adar
- Common Neighbors
- Preferential Attachment
- Resource Allocation
- Same Community
- Total Neighbors



Centrality & Importance

- Degree Centrality
- Closeness Centrality
- Harmonic Centrality
- Betweenness Centrality
- PageRank
- Personalized PageRank
- ArticleRank
- Eigenvector Centrality
- Hyperlink Induced Topic Search (HITS)
- Articulation Points
- Bridges
- Influence Maximization (CELF based on Greedy)



Supervised Machine Learning

- Node Classification
- Node Regression
- Link Prediction

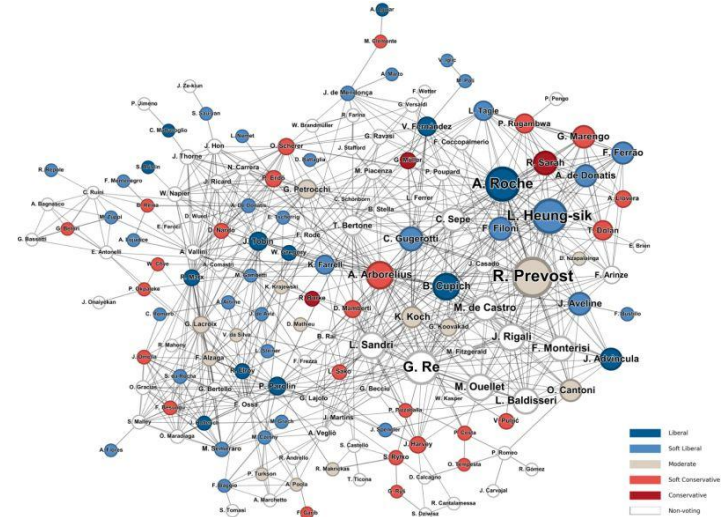


And More!

- Topological Sort
- Longest Path
- Pregel API (write your own algorithms)
- Scale Properties
- One Hot Encoding
- Split Relationships

<https://neo4j.com/blog/aura-graph-analytics/graph-analytics-basics/>

GDS – Exemplo Eleição do Papa



Em 8 de maio, antes da eleição do Papa Leão XIV (Cardeal Robert Prevost), um estudo da Universidade Bocconi já o apontava como o cardeal número 1.

Essa medida de influência, fruto da análise das conexões no Colégio Cardinalício, foi calculada usando a **centralidade de autovetor** (eigenvector centrality).

GDS – Exemplo Eleição do Papa

"Nosso ponto de partida é simples mesmo na Igreja, como em qualquer organização humana, os relacionamentos são importantes. Quanto mais conectado, ouvido e central um indivíduo estiver no fluxo de informações, maior a probabilidade de ele se tornar uma figura unificadora."

Giuseppe Soda

GDS – Exemplo Eleição do Papa

A equipe de pesquisa reconstruiu um modelo da "rede do Vaticano" usando três fontes principais:

- a. **Co-filiações oficiais** (dicastérios, comissões, conselhos, academias da Cúria Romana): esses dados revelam quem trabalha com quem e em quais contextos institucionais.
- b. **Linhas de consagração episcopal**: cada cardeal foi ordenado por outros, e essas "genealogias espirituais" constroem fortes laços de lealdade e reconhecimento.
- c. **Relações informais**: mapeadas por meio de fontes jornalísticas autorizadas, incluem afinidades ideológicas, relações de mentoria e participação em redes de clientelismo.

GDS – Exemplo Eleição do Papa

Status, informação e alianças: três chaves para o poder

Três critérios que determinam a "proeminência" de um cardeal na rede eclesiástica:

- a. Status, medido pela "**centralidade do autovetor**", recompensa os cardeais que estão conectados não apenas a muitos, mas aos mais influentes.
- b. Controle da informação, estimado usando a "**centralidade de intermediação**", identifica aqueles que atuam como centros entre diferentes grupos: pontes, em vez de pilares.
- c. Capacidade de construir coalizões, calculada usando um índice composto que combina agrupamento (que reflete o quanto um cardeal faz parte de um grupo coeso baseado na confiança e capaz de fornecer forte apoio) e influência direta (por meio da **centralidade do cardeal**, que mede quantas conexões diretas eles têm - quanto maior o grau, maior a influência e a popularidade) e papel estratégico (ou seja, até que ponto um cardeal pode atuar como uma ponte social graças à sua posição no centro das conversas ou conexões entre outros membros da rede, melhorando sua capacidade de construir alianças entre grupos).

GDS – Exemplo Eleição do Papa

Status, informação e alianças: três chaves para o poder

Top 5 por status

1. **Robert Prevost (moderado, EUA)**
2. Lazzaro You Heung-sik (liberal suave, Coreia do Sul)
3. Arthur Roche (liberal, Reino Unido)
4. Jean-Marc Aveline (liberal suave, França)
5. Claudio Gugerotti (liberal suave, Itália)

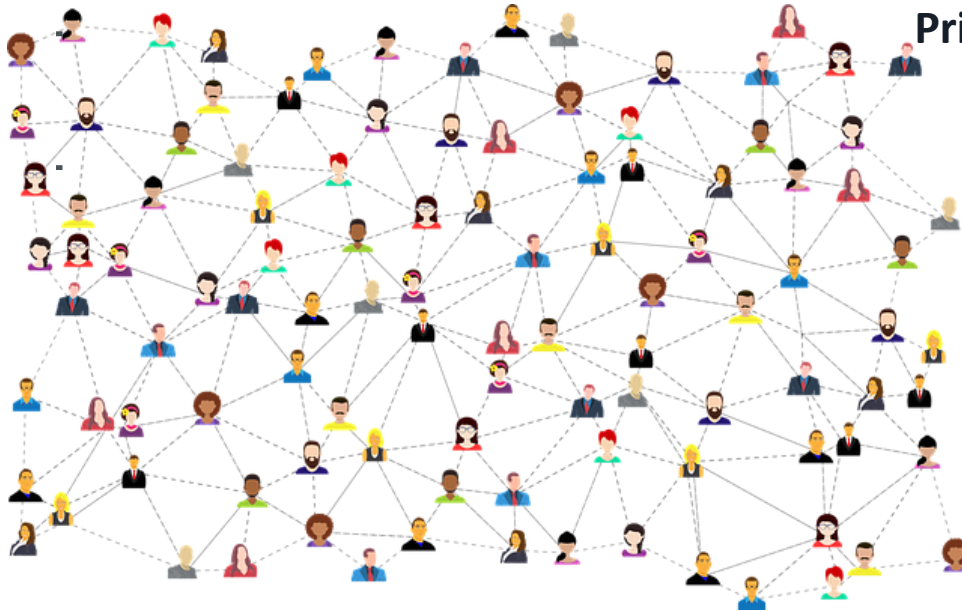
Top 5 para controle de informações

1. Anders Arborelius (conservador suave, Suécia)
2. Pietro Parolin (liberal, Itália)
3. Víctor Fernández (liberal, Argentina)
4. Gérald Lacroix (moderado, Canadá)
5. Joseph Tobin (liberal, EUA)

Top 5 para capacidade de construção de coalizão

1. Luis Antonio Tagle (liberal suave, Filipinas)
2. Ángel Fernández Artime (liberal suave, Espanha)
3. Gérald Lacroix (moderado, Canadá)
4. Fridolin Besungu (conservador suave, Congo)
5. Sérgio da Rocha (liberal suave, Brasil)

GDS – Centralidade



Principais perguntas respondidas:

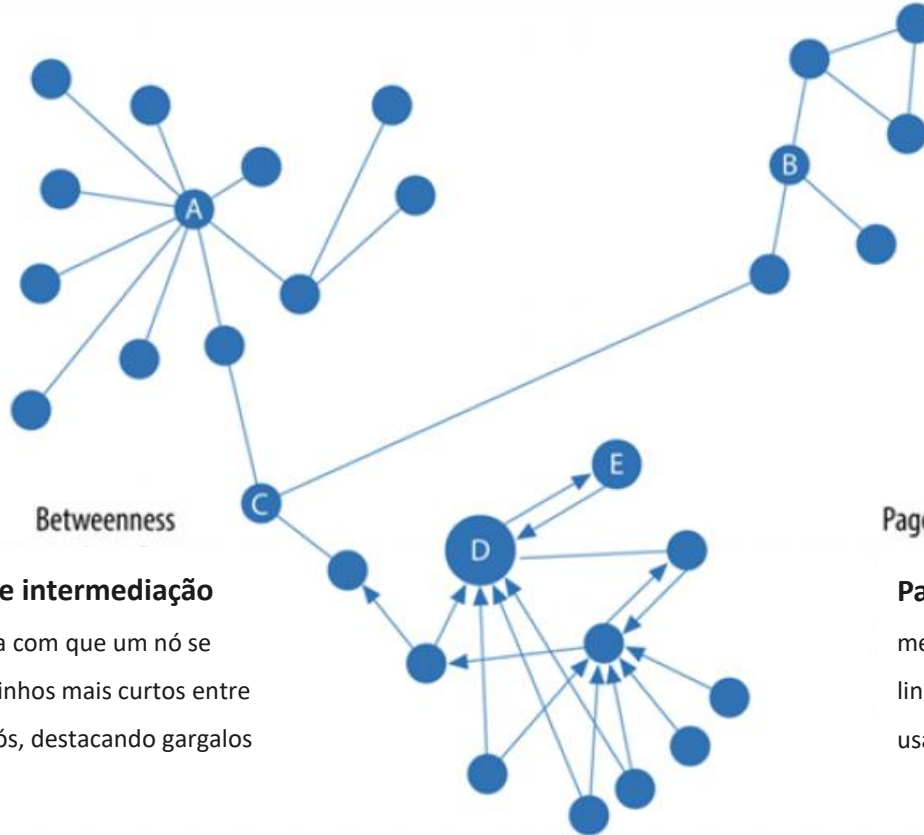
- Quais são os nós mais influentes ou críticos?
- Quais nós atuam como conectores, pontes ou centros principais?
- Onde se encontram os potenciais gargalos ou centros de atividade?
- Quais nós são mais facilmente acessíveis por outros?

GDS – Centralidade

Degree

Centralidade de grau

mede a importância com base simplesmente no número de conexões diretas que um nó possui; útil para identificar nós populares.



Closeness

Centralidade de Proximidade

Identifica os nós que possuem a menor distância média em relação a todos os outros nós.

Betweenness

Centralidade de intermediação

mede a frequência com que um nó se encontra nos caminhos mais curtos entre outros pares de nós, destacando gargalos ou pontes.

PageRank

PageRank

mede a influência com base na qualidade dos links recebidos de nós importantes; comumente usado para classificar relevância ou influência.

GDS – Centralidade

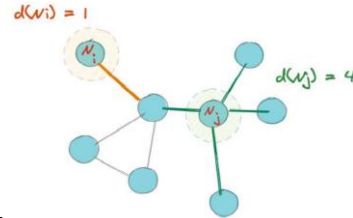
Centralidade de grau

atribui uma pontuação de importância baseada simplesmente no número de links mantidos por cada nó. Quantas conexões diretas de 'um salto' cada nó tem com outros nós da rede.

Às vezes é útil analisar o grau de entrada (número de links de entrada) e o grau de saída (número de links de saída) como medidas distintas, por exemplo, ao observar dados transacionais ou atividades da conta.

Exemplos:

- ✓ Para encontrar indivíduos muito conectados, indivíduos populares, indivíduos que provavelmente detêm a maior parte das informações ou indivíduos que podem se conectar rapidamente à rede mais ampla.



GDS – Centralidade

- Centralidade de intermediação - Betweenness

Cada nó recebe uma pontuação, com base no número de caminhos mais curtos que passam pelo nó. Os nós que se encontram com mais frequência nos caminhos mais curtos entre outros nós terão pontuações mais altas.

Esta medida mostra quais nós são “pontes” entre nós de uma rede.

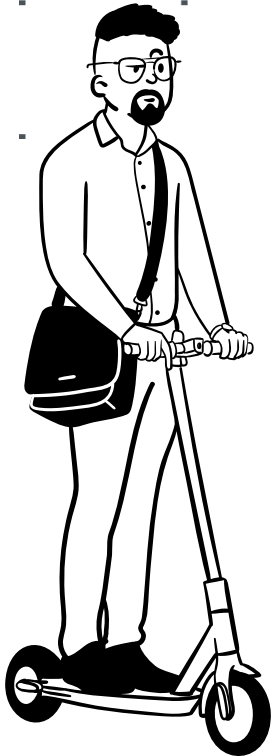
Útil para analisar a dinâmica da comunicação, mas deve ser usada com

Exemplos:

- ✓ Para encontrar os indivíduos que influenciam o fluxo em torno de um sistema.



GDS – Centralidade



- Centralidade de proximidade - Closeness

Pontua cada nó com base em sua “proximidade” com todos os outros nós da rede. Esta medida calcula os caminhos mais curtos entre todos os nós e, em seguida, atribui a cada nó uma pontuação com base na soma dos caminhos mais curtos.

Exemplos:

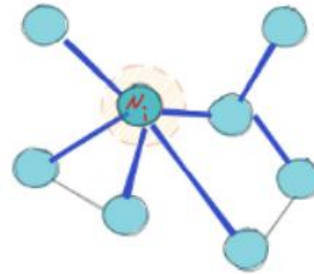
- ✓ Para encontrar os indivíduos que estão em melhor posição para influenciar toda a rede mais rapidamente.
- ✓ pode ajudar a encontrar boas 'emissoras', mas em uma rede altamente conectada, frequentemente se descobre que todos os nós têm uma pontuação semelhante. O que pode ser mais útil é usar a Proximidade para encontrar influenciadores em um único cluster.

GDS – Centralidade

- Centralidade de proximidade (closeness)

Closeness centrality

$$C_c(N_i) = \frac{|G| - 1}{\sum_{j=1}^{|G|} L_{ij}}$$



The Shortest path
from node i to all nodes

Length of the shortest path
between nodes i and j

Analisa a rapidez com que um nó atinge outros nós.

GDS – Centralidade

PageRank

Atribuindo aos nós uma pontuação com base em suas conexões e nas conexões de suas conexões.

Essa medida descobre nós cuja influência se estende além de suas conexões diretas na rede mais ampla.

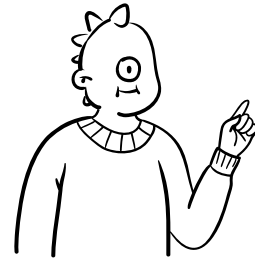
Pagerank centrality

$$C_{pr}(w_i) = D (D - \alpha A)^{-1} \mathbf{1}$$

$D = \begin{bmatrix} \text{ } & \text{ } \\ \text{ } & \text{ } \end{bmatrix}$

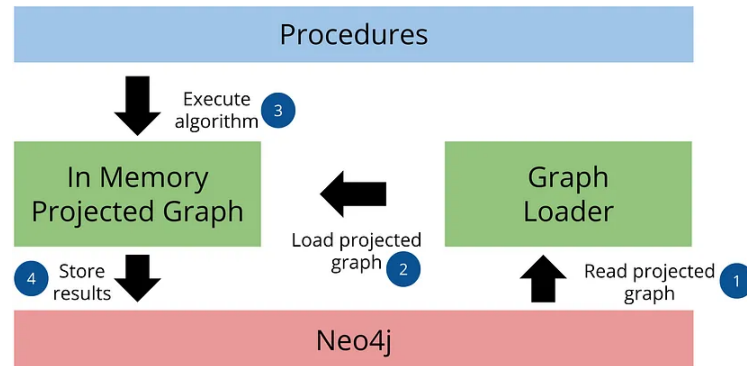
$D_{ii} = \max(d_{out}, 1)$

Damping factor that scale the graph topology effect on your measure.



Neo4J - Graph Projection

- a. Uma perspectiva do grafo armazenado
- b. Contém apenas informação analítica relevante
- c. Armazenado em memória utilizando compressão de dados
- d. Otimizado para operações de busca de propriedade ou relacionada a topologia do grafo.



GDS - Similaridade

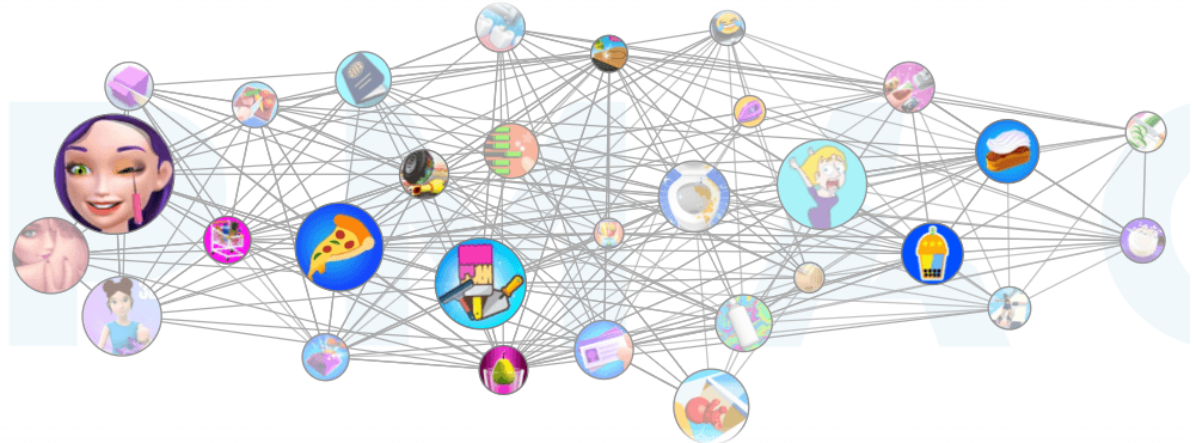
Similaridade entre atributos dos nós ou entre a topologia dos grafos.

APPLICATIONS
Super Salon ▾

GRAPH NODES

30

Min. similarity 18%



GDS - Similaridade

O coeficiente de **similaridade de Jaccard** mede a similaridade entre os conjuntos de vizinhos de dois vértices com base na interseção e na união dos elementos nesses conjuntos.

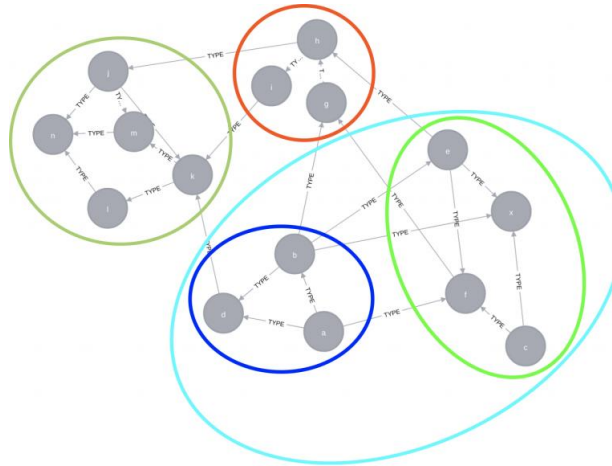
Por exemplo, num grafo social onde os vértices representam pessoas e as arestas representam amizades. Pode-se calcular a similaridade entre duas pessoas, A e B, obtendo um coeficiente variando de 0 a 1, onde 0 indica nenhuma similaridade (nenhum vizinho em comum) e 1 indica máxima similaridade (todos os vizinhos são os mesmos). Com isso:

- a) **Identificação de Relacionamentos** Próximos: identificar pares de vértices que compartilham conexões semelhantes. Isso pode ser útil em redes sociais para encontrar usuários com interesses semelhantes ou em redes de colaboração científica para identificar pesquisadores que trabalham em áreas relacionadas.
- b) **Recomendação de Amizades ou Conexões**: recomendar novas conexões. Por exemplo, em uma rede social, podemos sugerir novos amigos para um usuário com base na similaridade dos amigos existentes.
- c) **Detecção de Anomalias**: Vértices com baixa similaridade com seus vizinhos podem ser considerados anomalias na rede. Esses vértices podem representar outliers ou pontos de interesse especial que merecem investigação adicional.

GDS – Detecção de Comunidades

Usada para avaliar como os grupos de nós são agrupados ou particionados, bem como sua tendência a se fortalecer ou se separar.

O grafo original G , pode ser subgrafos / comunidades.



Community Detection

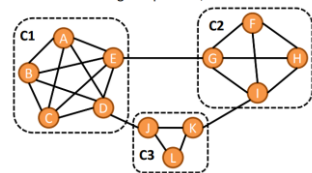
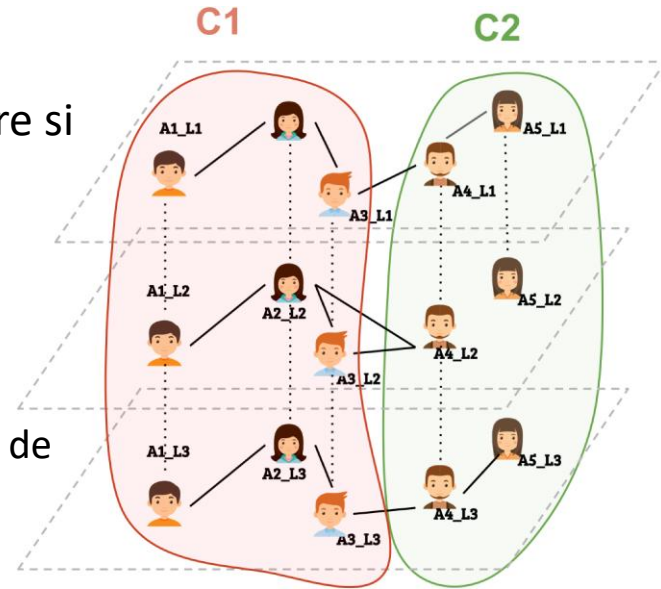
- Triangle Count
- Local Clustering Coefficient
- Connected Components (Union Find)
- Strongly Connected Components
- Label Propagation
- Louvain Modularity
- K-1 Coloring
- Modularity Optimization

GDS – Detecção de Comunidades

Oferece compreensão de quantos nós se relacionam entre si em grupos.

Exemplos:

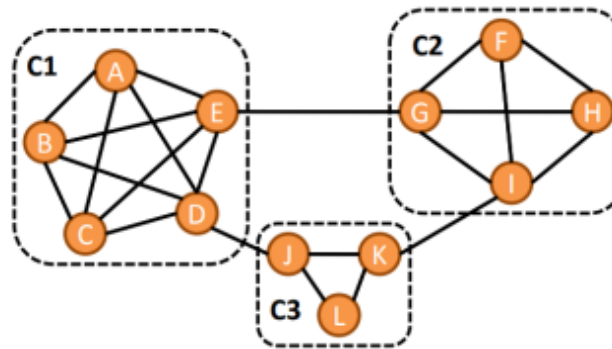
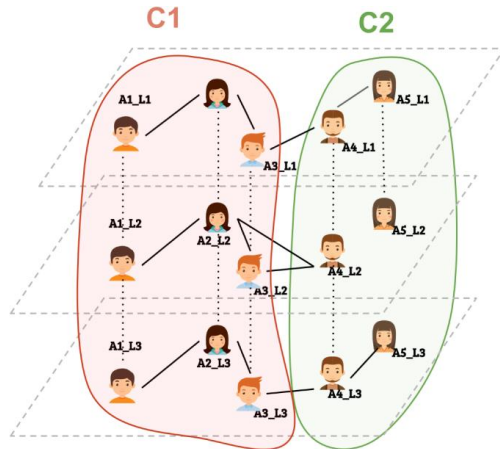
- ✓ Detectando contas interconectadas em uma rede de fraudes de cartão de crédito
- ✓ Fornecer grupos de funcionários com ideias semelhantes em uma rede de habilidades relacionadas



C1={A,B,C,D,E}
C2={F,G,H,I}
C3={J,K,L}

GDS – Detecção de Comunidades

A definição mais geral é baseada no princípio de que pares de nós são mais propensos a serem conectados se ambos forem membros da(s) mesma(s) comunidade(s), e menos propensos a serem conectados se não compartilharem comunidades.



C1={A,B,C,D,E}
C2={F,G,H,I}
C3={J,K,L}

GDS – Detecção de Comunidades

Girvan-Newman (2002)

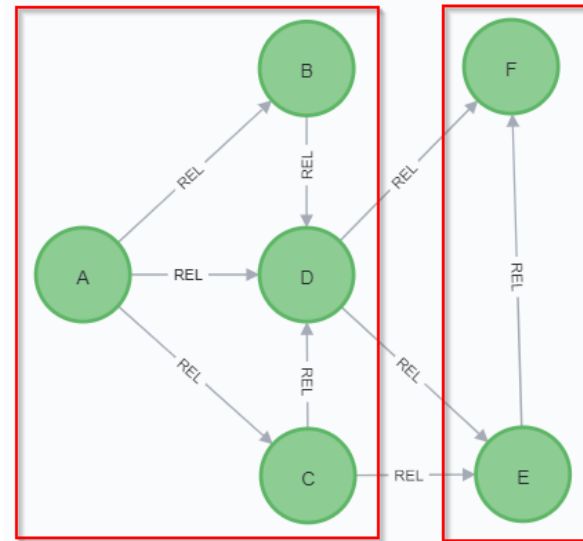
- um algoritmo comum usado para particionar clusters dentro de uma rede
- trabalha em duas etapas principais, repetindo até que não haja mais arestas:
 - a) Calcular a intermediação das arestas
 - b) Remover a aresta com a maior intermediação
- É importante notar que a intermediação das arestas deve ser recalculada toda vez que uma aresta é removida.

GDS – Detecção de Comunidades

Louvain Modularity (2008)

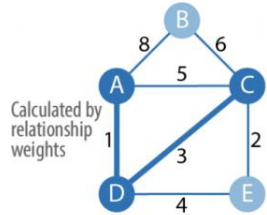
- avaliar o quanto mais densamente conectados são os nós dentro de uma comunidade, em comparação com o quão conectados estariam em uma rede aleatória.
- detecta duas comunidades [A, B, C, D] e [E, F], quando o grafo é considerado não direcionado e não ponderado.

Cada comunidade recebe um ID exclusivo



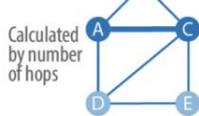
GDS – Caminhos - Pathfinding

Pathfinding Algorithms



Shortest Path

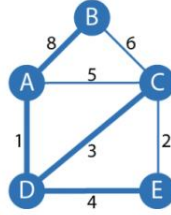
Shortest path between 2 nodes (A to C shown)



(A, B) = 8
(A, C) = 4 via D
(A, D) = 1
(A, E) = 5 via D
(B, C) = 6
(B, D) = 9 via A or C
And so on...

All-Pairs Shortest Paths

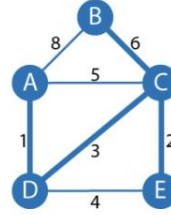
Optimized calculations for shortest paths from all nodes to all other nodes



Single Source Shortest Path

Shortest path from a root node (A shown) to all other nodes

Traverses to the next unvisited node via the lowest cumulative weight from the root



Minimum Spanning Tree

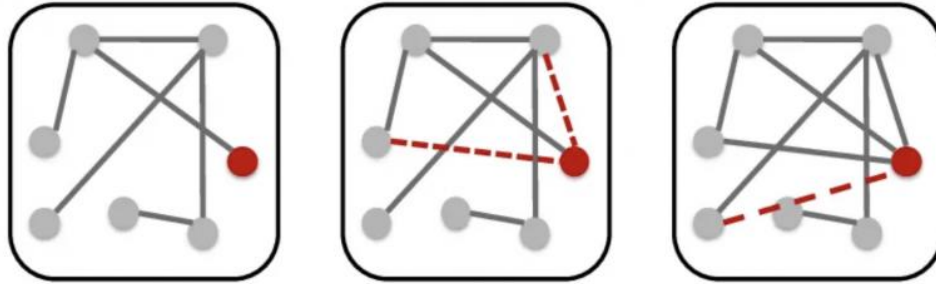
Shortest path connecting all nodes (A start shown)

Traverses to the next unvisited node via the lowest weight from any visited node

Exemplos:

- ✓ Explicando como dois membros de uma rede social estão relacionados
- ✓ Compreender a complexidade entre dois pontos de uma cadeia de abastecimento
- ✓ Encontrando caminhos biológicos comuns entre duas proteínas

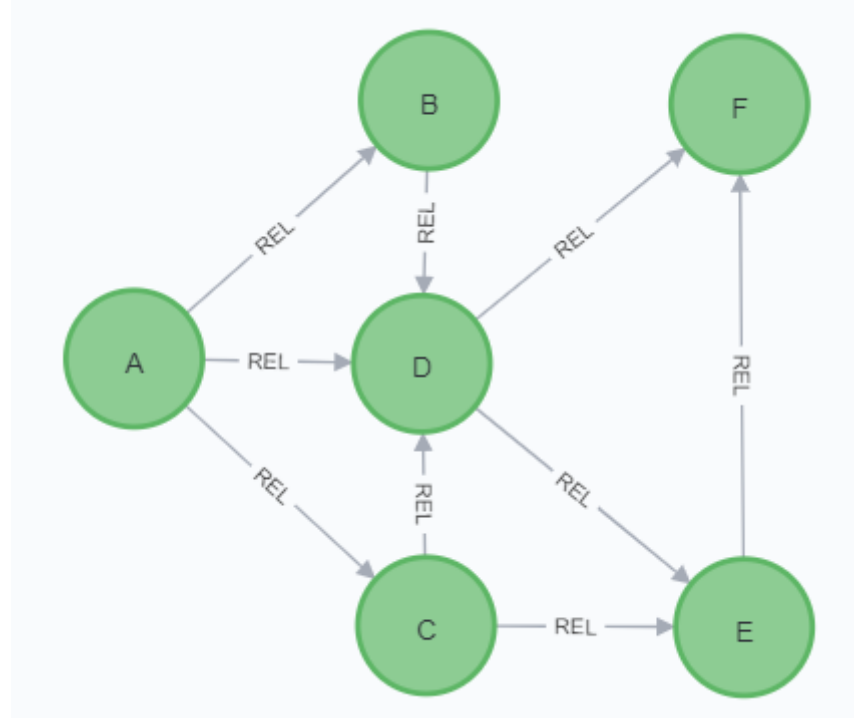
GDS - Predição de links



Técnica para tratamento de links ausentes, que consiste na aplicação de técnicas e métodos preditivos para prever links que não estão atualmente representados em uma rede.

GDS - Predição de links

- retornam uma pontuação numérica, frequentemente entre 0 e 1.
- Todas as consultas medem a proximidade do nó A de qualquer outro nó no grafo.



GDS - Predição de links

Adamic Adar

- calcula a proximidade de A para qualquer outro nó.
- A está mais próximo de D (pontuação de 2,35 aqui), porque D basicamente conecta todos os outros nós no grafo.

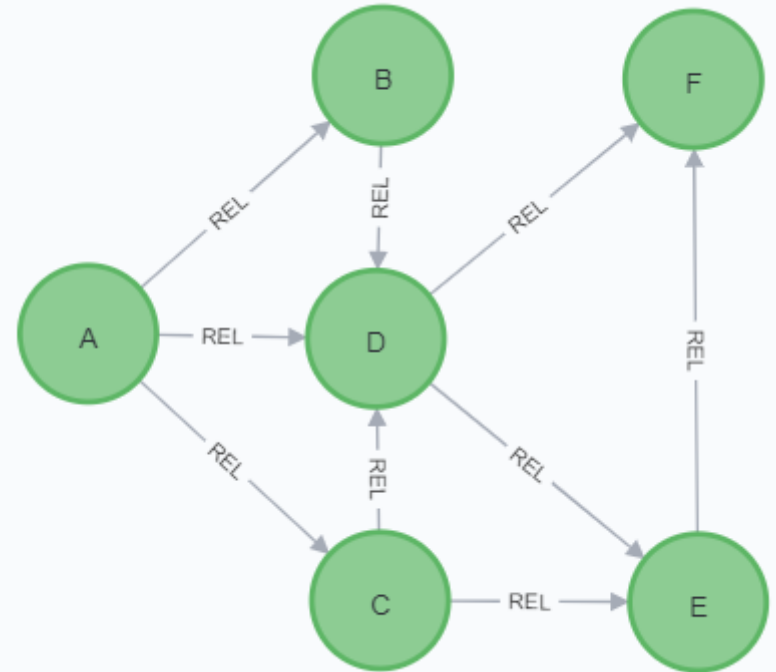
A -> D = 2,35

A -> E = 1,53

A -> F = 0,62

A -> B = 0,62

A -> C = 0,62



GDS - Predição de links

Preferential Attachment

- é uma medida usada para calcular a proximidade de nós, com base em seus vizinhos compartilhados.
- Um valor de 0 indica que dois nós não estão próximos, enquanto valores mais altos indicam que os nós estão mais próximos..

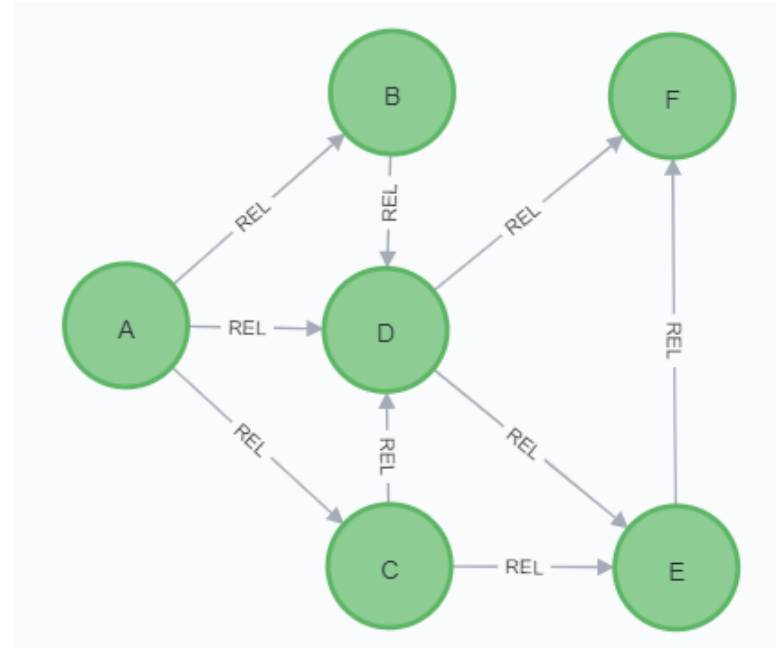
A -> D = 15

A -> E = 9

A -> F = 6

A -> B = 6

A -> C = 9



GDS - Predição de links

Common Neighbors .

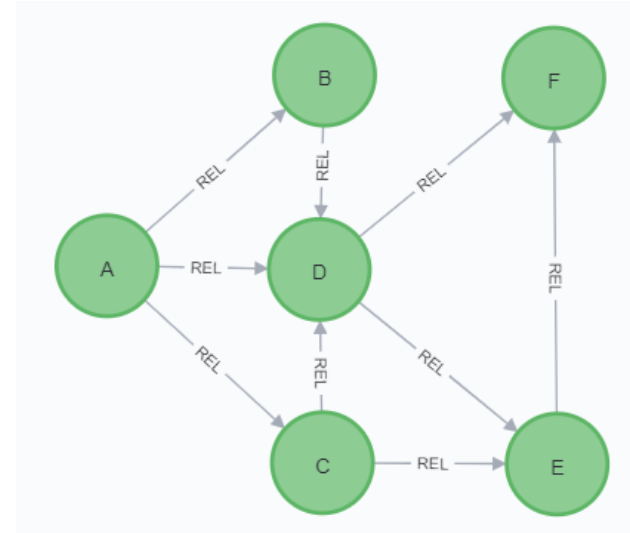
- Vizinhos comuns capta a ideia de que dois estranhos que têm um amigo em comum são mais propensos a serem apresentados do que aqueles que não têm nenhum amigo em comum.
- Um valor de 0 indica que dois nós não estão próximos, enquanto valores mais altos indicam que os nós estão mais próximos.

A e D se conectam diretamente e têm B e C como vizinhos comuns (isso é o que importa).

A e E não se conectam diretamente, mas também possuem dois nós em comum (D e C).

A e F têm apenas D em comum.

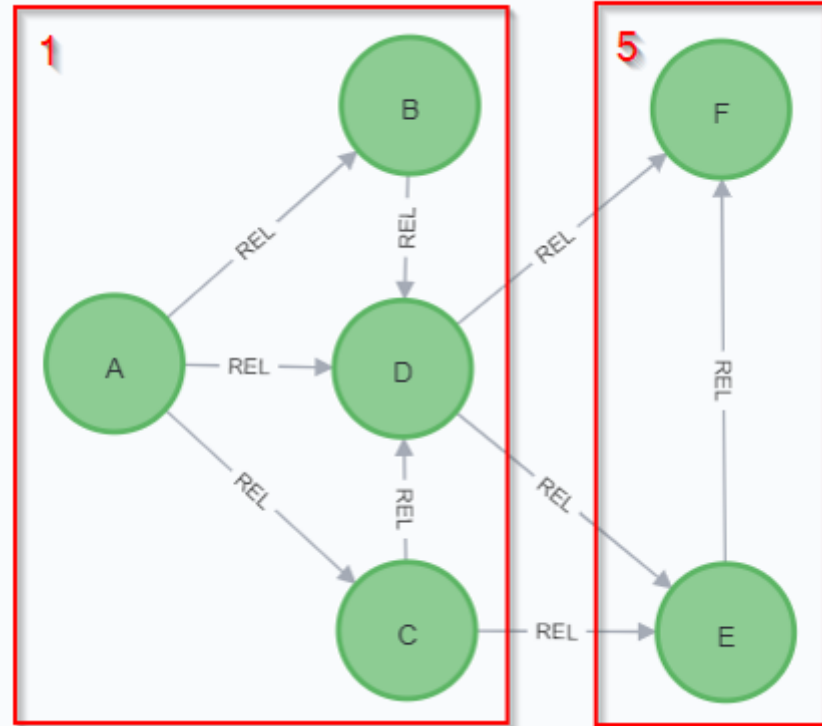
A e B se conectam diretamente e têm apenas D em comum.



GDS - Predição de links

Same Community

- é uma maneira de determinar se dois nós pertencem à mesma comunidade. Essas comunidades podem ser calculadas usando um dos algoritmos de detecção da Comunidade.
- Os nós B, C e D são encontrados na mesma comunidade com A (pontuação 1, para a comunidade 1). Mas não E e F (pontuação 0, para comunidade 5):





Hands On – Graph Data Science

Exemplos GDS

<https://sandbox.neo4j.com/>



Hands On – Case VArejo

Load dados_transacoes.txt
Resultado dos comandos

Neo4j – Desenvolvimento de Aplicações



HTTP Cypher request no Neo4j Browser

```
:POST /db/<database-name>/tx/commit {"statements":[
  {"statement":"CREATE (p:Person {firstName: $name}) RETURN p",
  "parameters":{"name":"Daniel"}}
]}
```

Em Python

```
from neo4j import GraphDatabase

# URI examples: "neo4j://localhost", "neo4j+s://xxx.databases.neo4j.io"
URI = "<URI for Neo4j database>"
AUTH = ("<Username>", "<Password>")

with GraphDatabase.driver(URI, auth=AUTH) as driver:
    driver.verify_connectivity()

# Get the name of all 42 year-olds
records, summary, keys = driver.execute_query(
    "MATCH (p:Person {age: $age}) RETURN p.name AS name",
    age=42,
    database_="neo4j",
)

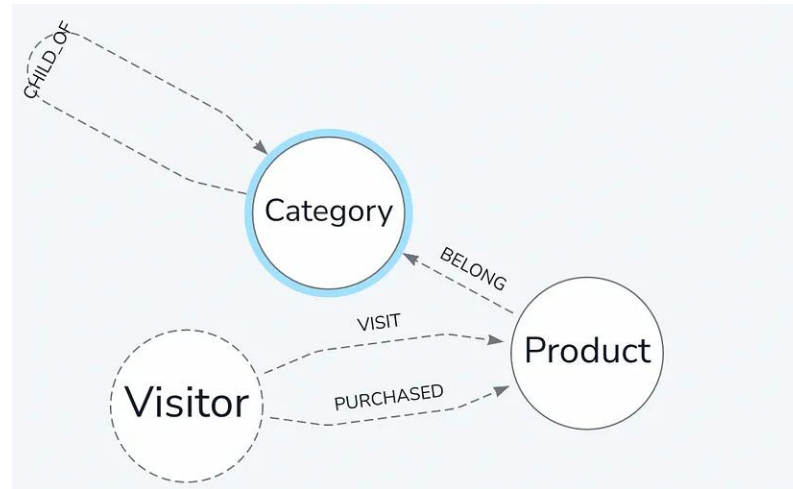
# Loop through results and do something with them
for person in records:
    print(person)
```

Neo4j – Desenvolvimento de Aplicações

Spark

```
spark.read.format("org.neo4j.spark.DataSource") \
  .option("url", "neo4j://localhost:7687") \
  .option("gds", "gds.pageRank.stream") \
  .option("gds.graphName", "myGraph") \
  .option("gds.configuration.concurrency", "2") \
  .load() \
  .show(truncate=False)
```

Bigquery e Neo4j





FIAP